

# Deploying Vortex FPGA development environment with Apptainer

Udit Subramanya, Rahul Raj Devaraju Nelamangala, Jeff Young, and Hyesoon Kim  
 {usubramanya3, rn84, jyoung9, hyesoon.kim}@cc.gatech.edu  
 Georgia Institute of Technology

**Abstract**—FPGA prototyping on shared HPC clusters is often challenged by complex toolchains, hidden dependencies, and inconsistent runtime environments. We present Vortex Apptainer, a containerized solution that encapsulates both vendor and custom FPGA toolchains alongside our open-source RISC-V GPGPU project, Vortex. Vortex Apptainer enables reproducible and rapid deployment across heterogeneous servers, significantly streamlining environment setup on shared HPC infrastructure.

**Index Terms**—Slurm, Apptainer, Portability, FPGA

## I. INTRODUCTION

Shared HPC clusters span multiple OS distributions, library versions, and vendor toolchains making environment setup a time-consuming and error-prone task. We observe that the microservice architecture is widely adopted within traditional software engineering. In this scheme, each service and all its dependencies are packaged into a lightweight, self-contained unit known as a container. Containerized microservices ensures consistent, reproducible environments that deploy and scale reliably across heterogeneous infrastructures. Using this strategy, we developed Vortex Apptainer [1]: a containerized solution to the open-source RISC-V GPGPU, Vortex [2]. We detail our design choices, report observed improvements, and discuss planned enhancements. This offers a blueprint for open-source hardware projects to containerize their own development workflows.

## II. BACKGROUND

### A. HPC Environment

Our Slurm-managed cluster consists of heterogeneous hardware nodes, including general-purpose high-performance nodes and accelerator nodes with FPGAs or GPUs. To support diverse evaluation platforms, nodes may run different OS versions. Although tools are organized under a consistent directory structure, users lack root privileges and cannot modify system-level configurations. These differences complicate portability and reproducibility. To address this, we aim to support deployment across any available FPGA-capable nodes, independent of their OS or hardware setup.

### B. Toolchain Requirements

Vortex, an open-source RISC-V GPGPU, has a complex toolchain even without the FPGA components. It includes the Portable OpenCL runtime, a customized fork of the

LLVM compiler to support custom ISAs, RISC-V cross-compilers, and a suite of open-source, cycle-accurate hardware simulation and synthesis tools (i.e., Yosys, Verilator), along with Ramulator for memory system modeling. To deploy the system on the Alveo U250 FPGA, we additionally require the Xilinx FPGA accelerator SDK, which includes the Vitis Core Development Kit (Vivado Design Suite and v++ compiler), the U250 platform packages, and the Xilinx Runtime (XRT) for FPGA kernel drivers. Combined, the entire toolchain exceeds 30 GB and has strict dependency requirements.

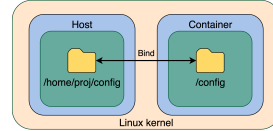


Fig. 1: Apptainer and Host File System Interaction. The option `--bind /home/proj/config:/config` describes a configuration where the Apptainer mounts the host path `/home/proj/config` at `/config`.

### C. Containerization

The Linux kernel exposes the same system-call interface, known as Application Binary Interface (ABI). Linux distributions use the same underlying Linux kernel but package very different collections of user-space tools, libraries, and default settings. Hence, a container can execute any Linux distribution if the binaries adhere to the host's ABI. Figure 1 shows how a container can be configured to mount host path with full read, write, execute permissions. By mounting host directories with full read-write-execute access, a container can run host executables, access I/O devices or modify host files. Table I describes the differences between two popular containerization tools. In multi-user HPC environments, Apptainer's rootless execution model and image isolation are preferred. We mitigate Apptainer's risk of large image footprints and lengthy build times through a strategic container architecture.

## III. PROBLEM STATEMENT

We have two objectives: (1) build the Vortex project & generate FPGA bitstream (2) run bitstream on FPGA hardware. Vortex Apptainer efficiently addresses the HPC environment and toolchain challenges to achieve both objectives.

| Feature      | Docker                                                        | Apptainer                                          |
|--------------|---------------------------------------------------------------|----------------------------------------------------|
| Image format | OCI-compliant tarball with multiple layers                    | Single compressed SIF file; simple to distribute   |
| Privileges   | Requires root/daemon                                          | Runs rootless by default; offers a fakeroot option |
| Caching      | Layer-by-layer cache; attempts to share layers between images | No cache; full rebuild each time                   |

TABLE I: Docker vs. Apptainer

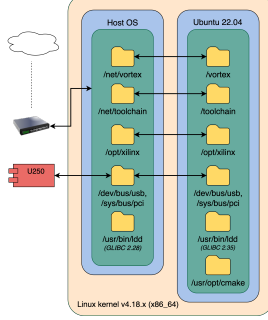


Fig. II: Vortex Apptainer Architecture tested on Intel Xeon Gold 6338 CPUs running Red Hat Enterprise Linux 8.10 connected to the Xilinx Alveo U250. Additionally, tested on Intel Xeon E5-2630 v4 CPUs running Ubuntu 22.04.5 LTS connected to Xilinx Alveo U50.

#### IV. VORTEX APPTAINER ARCHITECTURE

##### A. High Level Design

An Apptainer definition (`.def`) file is a plain-text manifest that specifies the base image, package dependencies, installation steps, environment variables, and on-launch script for the desired image. Our `.def` file uses an Ubuntu 22.04 base image. We iteratively determined the minimum additional dependencies required to satisfy our toolchain and FPGA vendor tools by running the build and run process until all requirements were satisfied. *Figure II* shows that we mounted Vortex’s source code & toolchain, FPGA vendor tools into the image, and the PCIe & USB devices that the FPGA exposes. The `apptainer build` command generates a SIF image from the `.def` file. Finally, our `./run_apptainer.sh` script runs the image and specified `bind` options as a container. On launch, the container presents an interactive shell for the developers to work in the development environment.

##### B. Design Justification

*Table II* rationalizes the decision to install certain aspects within the Apptainer image versus mount them at runtime. In general, we only install light-weight, permanent contents in the image. It is safest to precisely mirror mount paths between the host and container, so existing workflows can resolve paths within the container. We only took liberty mounting Vortex project paths because those are reconfigurable. All mounts are specified in `./run_apptainer.sh` and these can be changed without rebuilding the container.

| Aspect       | Decision | Rationale                                                                                                                                    |
|--------------|----------|----------------------------------------------------------------------------------------------------------------------------------------------|
| Base image   | Install  | Ubuntu 22.04 supplies GLIBC 2.34+ (needed by toolchain) without modifying the shared RHEL 8 host limited to GLIBC 2.28                       |
| Dev tools    | Install  | Tools (ie., <code>cmake</code> , <code>ccache</code> ) can be freely installed without polluting host environment                            |
| Project code | Mount    | Project files need to persist beyond container lifetime. Edits should not require a rebuild                                                  |
| Toolchains   | Mount    | Combined >30 GB; complex and slow installation without package manager support. Can still run ABI compatible executables                     |
| FPGA device  | Mount    | Linux’s “everything is a file” abstraction; <code>/dev</code> and <code>/sys</code> passthrough function seamlessly if drivers are available |

TABLE II: Summary of Vortex Apptainer Composition

#### V. EVALUATION

##### A. Achievements

- Vortex Apptainer successfully builds Vortex, generates bitstream, and executes bitstream on hardware
- Temporary changes are quick to iterate on within the container, without elevated privileges; rolling back changes is as simple as exiting the container
- Apptainer’s self-documenting `.def` files enhance transparency, collaboration, and version controlling
- Large language models understand Linux and containers very well resulting in effective AI assisted development

##### B. Shortcomings

- Compute-intensive FPGA bitstream generation must still be done on accelerator nodes that provide the proprietary Xilinx toolchain and license-server access
- A lack of domain expertise and discipline can lead to a build-up of technical debt in the container design

#### VI. CONCLUSION AND FUTURE WORK

We presented Vortex Apptainer, a containerized FPGA development environment designed to address the challenges of deploying complex toolchains on heterogeneous HPC clusters. By encapsulating all necessary dependencies and enabling direct hardware access, Vortex Apptainer facilitates reproducible and portable development for the Vortex RISC-V GPGPU platform. It has been validated across nodes with different operating systems and hardware configurations.

Although the current version of Vortex Apptainer is tailored to our specific host machine configuration, we believe the underlying methodology is broadly applicable to other environments. As part of future work, we plan to integrate Vortex Apptainer with continuous integration (CI) pipelines to automate and validate the FPGA build process, and to develop a customizable script for generating Apptainer images suited to diverse hardware and software setups

#### REFERENCES

- [1] Linux Foundation, “Apptainer documentation,” <https://apptainer.org/docs/>, 2024, accessed: 2025-05-09.
- [2] B. Tine, K. P. Yalamarthy, F. Elsabbagh, and K. Hyesoon, “Vortex: Extending the risc-v isa for gpgpu and 3d-graphics,” in *MICRO-54*, ser. MICRO ’21, 2021, p. 754–766.